

Use of the SQG⁺¹ model to reconstruct upper-ocean flows from sea surface height

PROJECT OUTLINE

Author: Jinchang Li

Advisor: Prof. Shafer Smith

Date: April 2, 2026

1. Introduction

Building upon Ryan's notes, I have developed code in both MATLAB and Python (using the JAX library for hardware-accelerated automatic differentiation) to perform the inversion process. The source code is available on [GitHub](#).

The primary objective is to find the surface quasi-geostrophic potential Φ^0 that minimizes the objective function:

$$\min_{\Phi^0} \sum \left(\hat{\eta}^s(\Phi^0) - \hat{\eta}^{\text{true}} \right)^2. \quad (1.1)$$

This is essentially the ℓ_2 norm of the surface sea height difference. Here the superscript $\cdot^s$ means evaluate at surface ($z = 0$) and $\hat{\cdot}$ is the fourier transform. In the SQG⁺¹ model. In the most compact form we can write

$$\hat{\eta}^s = \mathcal{N}(\hat{\Phi}^{0,s}) + \hat{\Phi}^{0,s} = \mathcal{N}^{\text{tot}}(\hat{\Phi}^{0,s}). \quad (1.2)$$

1.1. Forward Process

Given the $\Phi^{0,s}$ field. The process of computing the nonlinear operator $\mathcal{N}^{\text{tot}}$ and obtain the surface sea height is called the **Forward Process**.

2. General Structure

Currently I am in the phase of testing the codes and model. It is a two part process.

2.1. Data Preparation

First, a "true" surface potential field, $\Phi_{\text{true}}^{0,s}$, is established. This is obtained either from idealized synthetic fields or from outputs of SQG simulations provided by Prof. Smith.

2.2. Forward Simulation

Using the SQG⁺ model described above, the forward process generates a corresponding “true” sea surface height field, η^{true} , from $\Phi_{\text{true}}^{0,s}$. This synthetic SSH serves as the observational data target for the inversion.

2.3. Inversion Process

The inversion is the core of the project. The optimizer is initialized with a suitable first guess, $\Phi_{\text{guess}}^{0,s}$. To evaluate robustness, I often construct this guess by adding scaled white noise to the true field (or using pure noise for idealized tests):

$$\Phi_{\text{guess}}^{0,s} = \Phi_{\text{true}}^{0,s} + \delta \cdot W,$$

where δ is a scaling factor and W is white noise. A Quasi-Newton optimization algorithm (specifically L-BFGS, implemented natively in JAX) iteratively updates $\Phi^{0,s}$ to minimize the cost function defined in Eq. 1.2.

3. Specific Adjustments and Results

3.1. Case 1

To start, I make up some fields to test different optimization methods algorithms.

3.1.1. Different Optimizer

There is a `lsqnonlin` in short for Least Square Non-linear optimizer supported in MATLAB. Instead taking a scalar computed by Eq 1.2 as the objective, the objective is

$$\hat{\eta}^s(\Phi^0) - \hat{\eta}^{\text{true}}.$$

This solver relies on the Levenberg-Marquardt algorithm.

Benchmark runs demonstrated that optimizing the scalar ℓ_2 -norm objective using a Quasi-Newton method is generally much faster than the nonlinear least-squares approach. Furthermore, the L-BFGS algorithm within the Python JAX package works better than MATLAB’s `fminunc` (Unconstrained Minimization).

3.2. Case 2: SQG Simulation Data

Subsequent tests utilized full SQG simulation data provided by Prof. Smith, which outputs a surface buoyancy field. Under SQG theory, spectral variables decay exponentially with depth, mak-

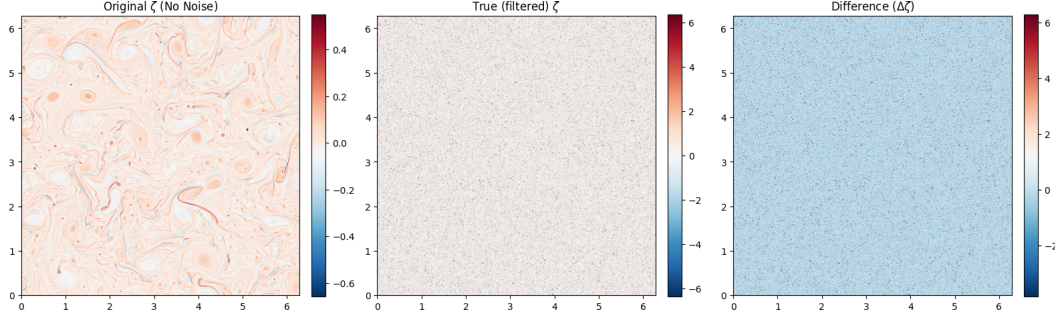


Figure 1: Grid size $N = 512$, scale factor $\delta = 0.05$. Comparison of the true field and the recovered SQG simulation field.

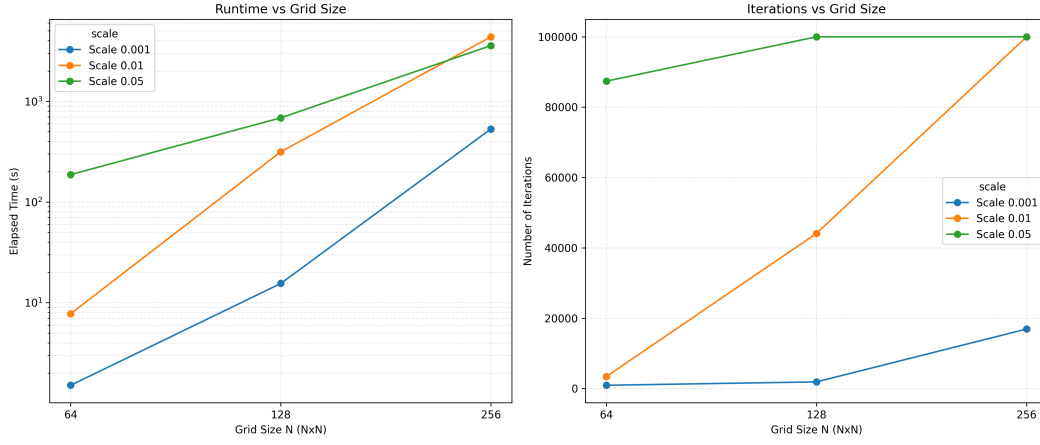


Figure 2: Influence of grid size and noise scale factor on the optimization run-time.

ing it straightforward to invert the surface buoyancy into a surface potential field, $\Phi_{\text{SQG}}^{0,s}$.

Since this field is purely governed by SQG dynamics, an ideal optimizer should perfectly recover $\Phi_{\text{SQG}}^{0,s}$. To explore the model's sensitivity, I perturbed the potential field to act as the "true" observational target:

$$\Phi_{\text{true}}^{0,s} = \Phi_{\text{SQG}}^{0,s} + \delta \cdot \|\Phi_{\text{SQG}}^{0,s}\|_{\infty} \cdot W, \quad (3.1)$$

where $\|\cdot\|_{\infty}$ denotes the maximum absolute amplitude, δ is a scaling factor controlling the noise level, and W is normalized filtered white noise. The optimizer is then initialized with the unperturbed SQG field:

$$\Phi_{\text{guess}}^{0,s} = \Phi_{\text{SQG}}^{0,s}.$$

3.2.1. Findings

The scale of the perturbation, δ , dramatically alters the resulting surface vorticity reconstruction. Setting $\delta = 0.05$ produces a highly noisy surface vorticity field (see Fig. 1), which in turn significantly increases the required optimization runtime (see Fig. 2).

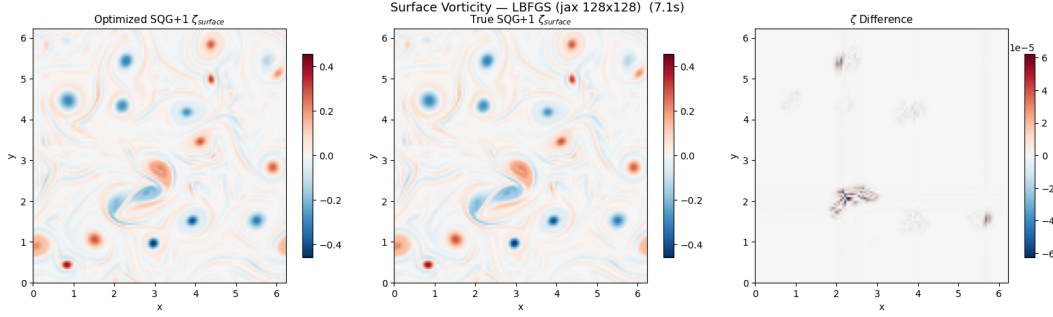


Figure 3: Grid Size $N = 128$, scale factor $\delta = 0.0001$.

However, if given enough run time. The optimization can still find the original field. Figure 3 shows a result with grid size 128×128 and a very small scale δ .

4. Further Approach

There are several potential options I would like to explore.

4.1. Physical Perturbations to SQG Simulations

Instead of simple white noise, I plan to introduce some physically meaningful perturbations representing submesoscale dynamics ($\Phi_{\text{submeso}}^{0,s}$) and internal gravity waves ($\Phi_{\text{wave}}^{0,s}$) to the SQG base field:

$$\Phi_{\text{true}}^{0,s} = \Phi_{\text{SQG}}^{0,s} + \Phi_{\text{submeso}}^{0,s} + \Phi_{\text{wave}}^{0,s}.$$

Passing this composite field through the forward model to generate SSH data would provide a more realistic scenario for testing the inversion scheme.

4.2. Validation via Channel Models

A limitation of the current approach is that the forward model used to generate the “true” SSH is the exact same SQG⁺ mathematical model used during the inversion process. So the result is that the exact potential $\Phi_{\text{true}}^{0,s}$ is mathematically guaranteed to yield an objective function of exactly 0. So the true field is actually, at least one of, the optimal solution. Regardless of the applied perturbations, a optimizer will predictably converge to $\Phi_{\text{true}}^{0,s}$ under these ideal conditions just a matter of time.

To evaluate the true physical validity of the SQG⁺ reconstruction, it is crucial to test the inversion against independent, physically complex simulation data. Tatsu has prepared me a Channel model run on primitive equations that generates both SSH and I can use the geostrophic surface potential as the initial guess and perform the inversion process **only**. Because the SSH and surface potential in a full primitive equation model are not strictly bound by simplified SQG⁺ con-

straints. It is expected that the reconstruction surface velocity field would not match the original field exactly.

4.3. Alternating the Objective

Currently we are using the ℓ_2 -norm of the difference between the reconstruction and true surface sea height as in Eq 1.2. Would it be more efficient to actually minimize the ℓ_2 -norm of the difference between the reconstruction and true surface vorticity? That is

$$\min_{\Phi^{0,s}} \sum \left(\hat{\zeta}^s(\Phi^{0,s}) - \hat{\zeta}^{\text{true}} \right)^2 \quad (4.1)$$

Or the surface strain.

4.4. Filtered Data

I saw in Ryan's notebook there is a low-pass filter :

```
1 lowpass_filter = np.ones(Kamat_cpkm.shape);
2 pass_k = 1/30 # this is the filter scale
3 lowpass_filter = np.array( np.exp(-(Kamat_cpkm-pass_k)**2*20) ) # This is
   the shape, I made this up from Smith 2002
4 lowpass_filter[Kamat_cpkm<pass_k] = 1
```

I am still not quite clear about how efficient this filter. I know it is mainly used to attenuate instrument noise and unbalanced submesoscale motions so that they won't adhere the SQG dynamics, i.e. the third and fourth term below.

$$u_{ocean} = u_{qg} + u_{agg,non-wave} + u_{wave} + u_{noise}$$

However I am also interested in other way of accomplishing similar target.